

Model-based RL

Samsung AI Expert

Jul 11, 2025

Giho Kim

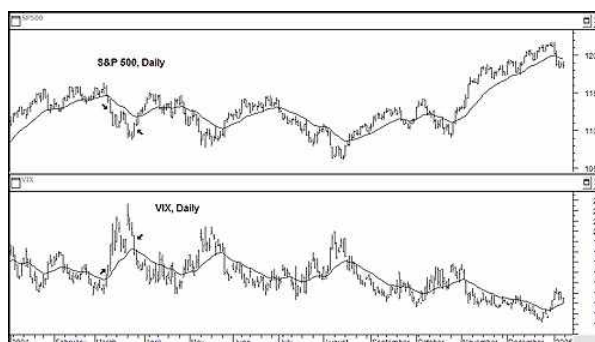
Model-based RL - Review

So far, we have learned **model-free** RL algorithms, i.e., learning policy/value function were done without model info:

$$s_{t+1} \sim p(\cdot | s_t, a_t), \quad r_t = r(s_t, a_t).$$

What is p ?

→ law of physics, artificial rules, etc.



Model-Based RL - Review

Model-based RL - Review

Can we **learn** p ?

simplest case : $s_{t+1} = s_t + f(s_t, a_t)$

Assume we have a large number of transition samples (s_j, a_j, s'_j) from the **true** transition dynamics f .

Then, we may learn a parametrized model f_θ which is **close** to f .

How?

Model-based RL - Review

Given a batch $B = \{(s_j, a_j, s'_j)\}_{j=1}^N$, we construct a loss as follows:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{j=1}^N \|s_{t+1} - s_t - f_{\theta}(s_t, a_t)\|^2$$

and update θ by gradient-based algorithms.

What can we do if we have a good model?

Model-based RL - Review

One can apply some well-known methods in control theory, such as **model predictive control(MPC)**:

$$A_t^{(H)} = \max_{(a_t, \dots, a_{t+H-1})} \sum_{t'=t}^{t+H-1} r(\hat{s}_{t'}, a_{t'})$$

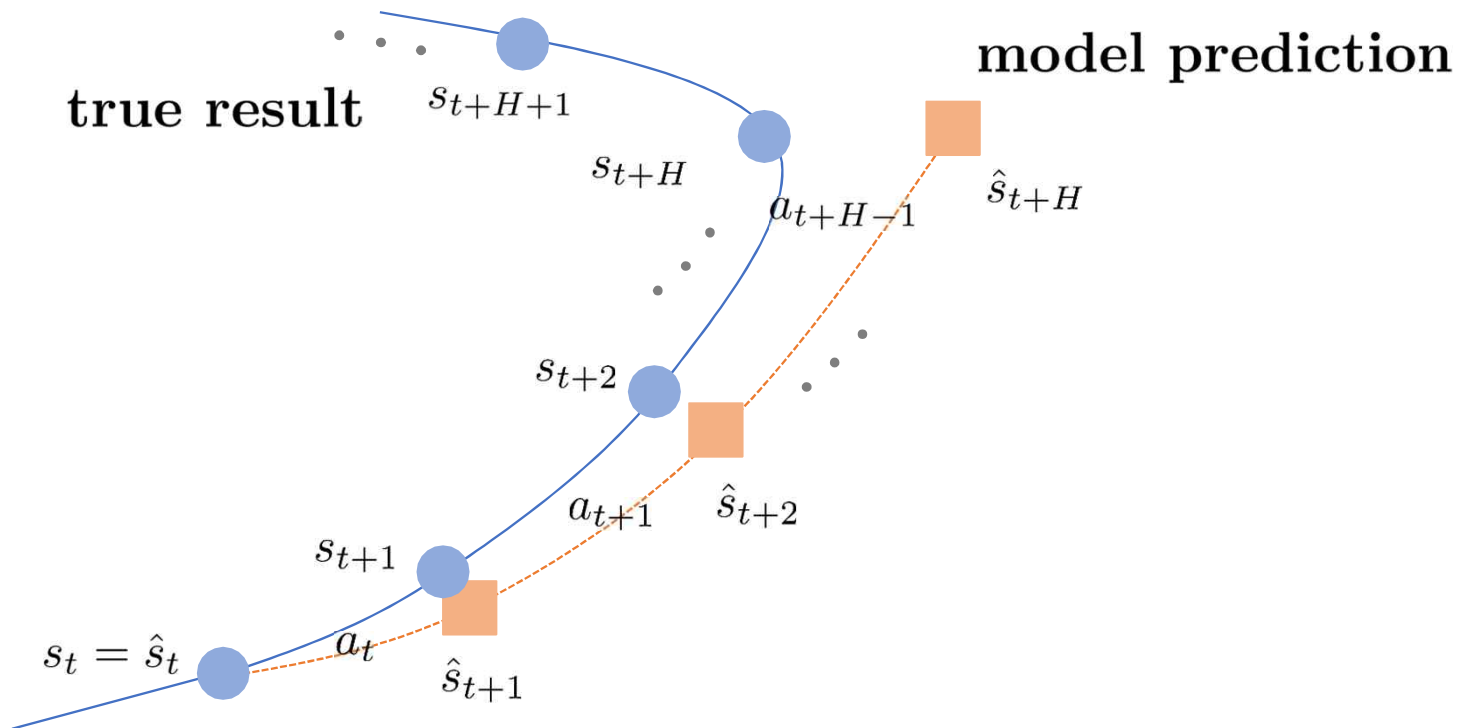
where

$$\hat{s}_t = s_t,$$

$$\hat{s}_{k+1} = \hat{s}_k + f_\theta(\hat{s}_k, a_k), \quad k = t, \dots, t + H - 1.$$

In this case, we will use a simple algorithm so called **random sampling shooting method**.

Model-based RL - Review



Random Shooting Method

Random sampling shooting

1. Generate N action sequences randomly.

$$\{(a_t^{(i)}, \dots, a_{T+H-1}^{(i)}) \mid i = 1, \dots, N\}$$

2. For each action sequence, compute the predicted state trajectory:

$$\{\hat{s}_t^{(i)}, \dots, \hat{s}_{T+H-1}^{(i)}\}$$

and the corresponding cumulative cost: $c^{(i)} := \sum_{k=t}^{T+H-1} c(\hat{s}_k^{(i)}, a_k^{(i)})$.

3. Choose the action sequence with the least $c^{(i)}$ as the solution:

Random Shooting Method

Random sampling shooting

1. Generate N action sequences randomly.

$$\{(a_t^{(i)}, \dots, a_{T+H-1}^{(i)}) \mid i = 1, \dots, N\}$$

- High sample complexity of random shooting due to uniform exploration

2. For each action sequence, compute the predicted state trajectory:

- Necessity of focused exploration in promising regions

$$\{\hat{s}_t^{(i)}, \dots, \hat{s}_{T+H-1}^{(i)}\}$$

and the corresponding cumulative cost: $c^{(i)} := \sum_{k=t}^{T+H-1} c(\hat{s}_k^{(i)}, a_k^{(i)})$.

3. Choose the action sequence with the least $c^{(i)}$ as the solution:

Cross Entropy Method

Cross-entropy method

1. Sample N action sequences from the current distribution $\mathcal{N}(\mu, \Sigma)$:

$$\{(a_t^{(i)}, \dots, a_{T+H-1}^{(i)}) \mid i = 1, \dots, N\}.$$

2. For each action sequence, compute the predicted state trajectory:

$$\{\hat{s}_t^{(i)}, \dots, \hat{s}_{T+H-1}^{(i)}\}$$

and the corresponding cumulative cost: $c^{(i)} := \sum_{k=t}^{T+H-1} c(\hat{s}_k^{(i)}, a_k^{(i)})$.

3. Update the distribution parameters (μ, Σ) using the top M elite sequences.

Advanced Model-based RL

combining with model-free RL?

Ex) Dyna Q learning¹

Initialize $Q(s, a)$ and $Model(s, a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$

Do forever:

- (a) $S \leftarrow$ current (nonterminal) state
- (b) $A \leftarrow \epsilon$ -greedy(S, Q)
- (c) Execute action A ; observe resultant reward, R , and state, S'
- (d) $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$
- (e) $Model(S, A) \leftarrow R, S'$ (assuming deterministic environment)
- (f) Repeat n times:
 - $S \leftarrow$ random previously observed state
 - $A \leftarrow$ random action previously taken in S
 - $R, S' \leftarrow Model(S, A)$
 - $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$

¹Sutton, Richard S. "Dyna, an integrated architecture for learning, planning, and reacting." *ACM Sigart Bulletin* 2.4 (1991): 160-163.

Advanced Model-based RL

more advanced Model-based RL algorithms?

Ex) World Models (Ha, Schmidhuber, 2018)

At each time step, our agent receives an **observation** from the environment.

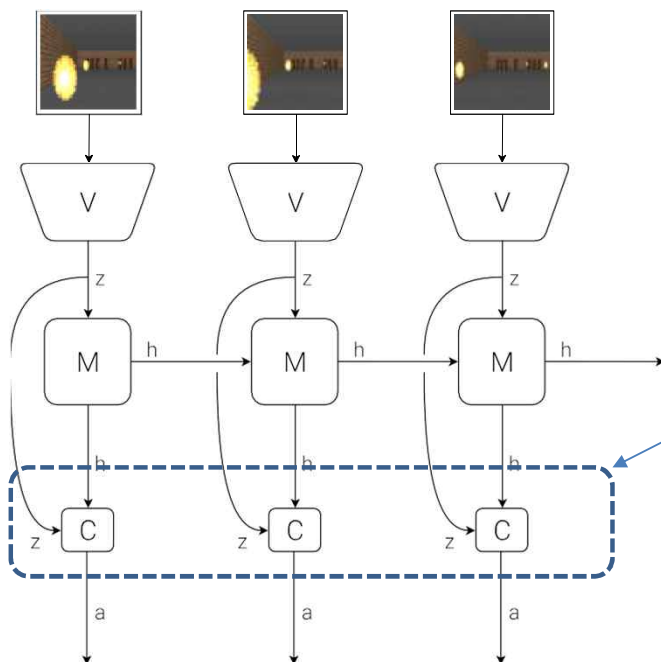
World Model

The **Vision Model (V)** encodes the high-dimensional observation into a low-dimensional latent vector.

The **Memory RNN (M)** integrates the historical codes to create a representation that can predict future states.

A small **Controller (C)** uses the representations from both V and M to select good actions.

The agent performs **actions** that go back and affect the environment.



actor-critic in Dreamer²

<https://worldmodels.github.io/>

²Hafner, Danijar, et al. "Dream to Control: Learning Behaviors by Latent Imagination." *International Conference on Learning Representations*.

Model-based RL - Implementation

Model-based RL - Implementation

```
1  class TransitionMemory:
2      def __init__(self, state_dim, act_dim, maxlen=20000):
3          ...
4      def append(self, state, act, next_state):
5          self.data.append((state, act, next_state))
6
7      def sample_batch(self, size):
8          ...
9          return (state_batch, act_batch, next_state_batch)
```

neural network : trained in supervised manner

⇒ only (s, a, s') needed! (**known reward function**)

Model-based RL - Implementation

```
1  class TransitionModel(nn.Module):
2      def __init__(self, state_dim, act_dim, hidden1, hidden2):
3          super(TransitionModel, self).__init__()
4          self.state_dim = state_dim
5          self.act_dim = act_dim
6          self.fc1 = nn.Linear(state_dim + act_dim, hidden1)
7          self.fc2 = nn.Linear(hidden1, hidden2)
8          self.fc3 = nn.Linear(hidden2, state_dim)
9
10     def forward(self, state, act):
11         x = torch.cat([state, act], dim=1)
12         x = F.relu(self.fc1(x))
13         x = F.relu(self.fc2(x))
14         delta = self.fc3(x)
15         next_state = state + delta
16
17     return next_state
```

neural network approximation of deterministic dynamics

Q. What if the transitions are stochastic?

Model-based RL - Implementation

```
1  class ModelBasedAgent:
2      def __init__(self, ...):
3          ...
4
5      def train(self, batch_size):
6          self.model.train()
7          (state_batch, act_batch, next_state_batch) = self.memory.sample_batch(batch_size)
8          state_batch = torch.tensor(state_batch).float()
9          act_batch = torch.tensor(act_batch).float()
10         next_state_batch = torch.tensor(next_state_batch).float()
11
12         prediction = self.model(state_batch, act_batch)
13         loss_ftn = MSELoss()
14         loss = loss_ftn(prediction, next_state_batch)
15         self.optimizer.zero_grad()
16         loss.backward()
17         self.optimizer.step()
18         loss_val = loss.detach().numpy()
19         return loss_val
```

Remark. This is a supervised learning, so we may try to measure validation error!

Model-based RL - Implementation : Shooting Method

Random sampling shooting

1. Generate N action sequences randomly.

$$\{(a_t^{(i)}, \dots, a_{T+H-1}^{(i)}) \mid i = 1, \dots, N\}$$

2. For each action sequence, compute the predicted state trajectory:

$$\{\hat{s}_t^{(i)}, \dots, \hat{s}_{T+H-1}^{(i)}\}$$

and the corresponding cumulative cost: $c^{(i)} := \sum_{k=t}^{T+H-1} c(\hat{s}_k^{(i)}, a_k^{(i)})$.

3. Choose the action sequence with the least $c^{(i)}$ as the solution:

Model-based RL - Implementation : Shooting Method

```
1 def solve_random_shooting_opt(self, state, H, N):
```

```
2     scores = np.zeros(N)
```

```
3     action_sequences = self.ctrl_range * (2. * np.random.rand(H, N, dimA) - 1.)
```

```
4  
5     states = np.tile(state, (N, 1))
```

```
6     for t in range(H):
```

```
7         actions = action_sequences[t]
```

```
8         scores += self.reward_model(states, actions)
```

```
9         s = torch.tensor(states).float()
```

```
10        a = torch.tensor(actions).float()
```

```
11        next_s = self.model(s, a)
```

```
12        states = next_s.detach().numpy()
```

```
13  
14     best_seq = np.argmax(scores)
```

```
15  
16     return action_sequences[best_seq]
```

generation of multiple control trajectories at once

simultaneous computation of predictions & cost along paths

Model-based RL - Implementation : Cross Entropy Method

Cross-entropy method

1. Sample N action sequences from the current distribution $\mathcal{N}(\mu, \Sigma)$:

$$\{(a_t^{(i)}, \dots, a_{T+H-1}^{(i)}) \mid i = 1, \dots, N\}.$$

2. For each action sequence, compute the predicted state trajectory:

$$\{\hat{s}_t^{(i)}, \dots, \hat{s}_{T+H-1}^{(i)}\}$$

and the corresponding cumulative cost: $c^{(i)} := \sum_{k=t}^{T+H-1} c(\hat{s}_k^{(i)}, a_k^{(i)})$.

3. Update the distribution parameters (μ, Σ) using the top M elite sequences.

Model-based RL - Implementation : Cross Entropy Method

How many elite samples?

generation of multiple control trajectories at once

```
def solve_cross_entropy_opt(self, state, H, N, rho=0.2, tol=1e-2, max_it=100):  
    Ne = int(rho * N)  
    ...  
    while np.max(sigma) >= tol and trial < max_it:  
        action_sequences = truncnorm.rvs(...)   
        for t in range(H):  
            .....  
            indices = np.argsort(scores)[-Ne:]  
            ...  
            elite_samples = action_sequences[:, indices, :]  
  
            mu += self.alpha * (np.mean(elite_samples, axis=1, keepdims=True) - mu)  
            sigma += self.alpha * (np.std(elite_samples, axis=1, keepdims=True) - sigma)  
            trial += 1  
    ...
```

pick out elite samples

update the distribution

Model-based RL - Implementation

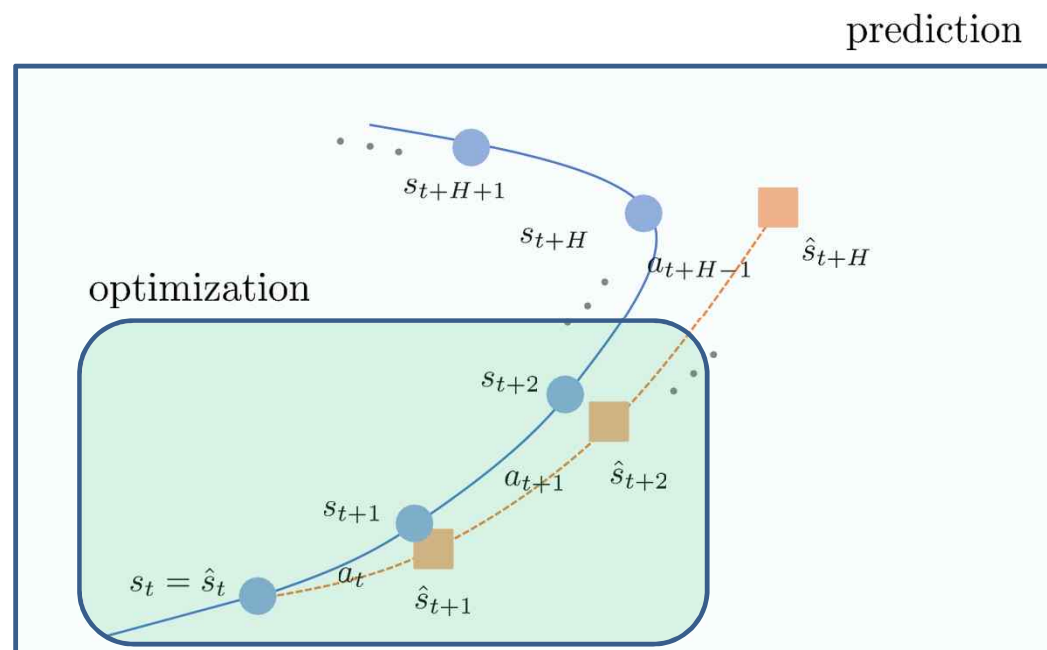
Some observations on random-shooting method

- Increasing # of trajectories : relatively less burden
- Increasing **horizon length** : **critical** to speed!
 - small horizon length $\implies$ low quality of resulting control
 - long horizon length $\implies$ high computational burden

In general, MPC is a computational bottleneck in complex, high-dimensional domains.

Model-based RL - Implementation

Any other option?



enabling exhaustive search in **control horizon**

Model-based RL - Experiment

Model-based RL – Experiment

OpenAI Gym Pendulum-v0

- environment file modified to allow simultaneous evaluation of multiple cost functions
- Test both random-shooting method & cross-entropy method!

